

Arrays

ADT • C++ Essentials • Pointers • Dynamic Memory • Dynamic Arrays • Rule of Three

1. Abstract Data Types (ADT)

An ADT is a description of WHAT a data type can do — its operations and rules — without saying HOW it is implemented internally. Think of it as a contract.

Data Type vs Data Structure vs ADT: these three are often confused.

Term	What it means	Example
Data Type	A category of values a variable can hold	int, char, bool
ADT	A set of operations + rules, hiding implementation	Stack ADT: push, pop, top, isEmpty
Data Structure	A concrete way of organizing and storing data to implement operations efficiently.	Array-based stack, Linked-list-based stack

Example — Stack ADT:

- Operations exposed to the user: push(x), pop(), top(), isEmpty(), size().
- The user never needs to know if it's built using an array or a linked list.
- A C++ class can be used to implement an ADT through encapsulation.

• If asked 'why use an ADT?' → answer: abstraction + encapsulation → implementation can change later without breaking code that uses it.

2. C++ Language Essentials Needed for Data Structures

2.1 Struct vs Class

- struct → members are public by default.
- class → members are private by default.
- Both can have constructors, destructors, and member functions — used interchangeably to build ADTs/nodes.

2.2 Pass by Value vs Pass by Reference vs Pass by Pointer

Method	Syntax	Can modify original?	Copies data?
By value	void f(int x)	No	Yes (full copy)
By reference	void f(int &x)	Yes	No
By pointer	void f(int *x)	Yes (via *x)	No (copies the address only)

- Data structures pass nodes/objects by reference or pointer to avoid expensive copying and to allow modification — this is why linked list functions take Node* or Node&.

3. Pointers Revisited

3.1 Variables, Memory, and Addresses

Every variable has two things: a VALUE and an ADDRESS (where it lives in memory).

```
int x = 10;
cout << x; // 10      → the value
cout << &x; // 0x61ff08 → the address
```

3.2 What Is a Pointer?

A pointer is a variable that stores an address instead of a normal value.

```
int x = 10;
int* p = &x; // p stores the ADDRESS of x
cout << p;   // prints x's address
cout << *p;  // prints 10 (dereferencing → 'value at that address')
```

Expression	Meaning
p	The address stored inside the pointer
&p	address of the pointer variable itself.
*p	The value living at that address (dereference)
&x	The address of variable x (address-of)
*p = 50;	Go to that address and overwrite the value → x becomes 50

3.3 Pointer Types & nullptr

- A pointer's type (int*, double*, Node*) tells the compiler how to interpret the bytes at that address.
- An uninitialized pointer holds a garbage address — dereferencing it is undefined behavior (a common crash cause).
- Always initialize a pointer that doesn't yet point anywhere to nullptr, and check before dereferencing.

```
int* p;           // DANGEROUS — garbage address
int* q = nullptr; // SAFE — points to nothing
if (q != nullptr) { cout << *q; }
```

- 'Why did this crash?' answers are almost always: dereferencing nullptr, dereferencing an uninitialized pointer, or using a pointer after delete (dangling pointer).

4. Dynamic Memory Allocation

4.1 Why Dynamic Memory?

Regular variables (int x;) are allocated on the STACK at compile time — their size must be known in advance. But often we don't know how much memory we need until the program runs (e.g., array size entered by the user). The HEAP solves this: memory requested at runtime using new.

4.2 new and delete

```
int* p = new int;    // one int on the heap
*p = 25;
delete p;           // release it

int* arr = new int[10]; // array of 10 ints on the heap

delete[] arr;       // must use delete[] for arrays
```

Allocated with	Must release with	Rule
new	delete	Single object
new[]	delete[]	Array — mismatching new/delete is undefined behaviour

4.3 Memory Leaks

A memory leak = allocated heap memory that becomes unreachable because nothing points to it anymore (and it was never deleted). It stays reserved for the whole program run, wasting memory.

```
int* p = new int(10);
p = new int(20); // BUG: old block (10) is now leaked — no pointer to it left
// Fix: delete p; BEFORE reassigning p
```

4.4 Stack vs Heap — the mental model

This is the single most useful picture for scenario questions: a pointer variable itself always lives on the STACK; what it points to may live on the STACK (if it points to a local variable) or the HEAP (if created with new).

Code	Stack holds	Heap holds	Meaning
int a = 25;	a = 25	—	Plain variable, no pointer involved
int* age;	age = garbage	—	Pointer declared but not initialized — unsafe

Code	Stack holds	Heap holds	Meaning
<code>int* age = new int(25);</code>	<code>age = 0x5000</code>	<code>0x5000 → 25</code>	age holds the address of a heap integer
<code>int a=25; int* age=&a;</code>	<code>a = 25, age = address of a</code>	—	Only ONE integer exists (a); age just points to it, nothing is on the heap
<code>Student s1(20);</code>	s1 object (contains age = 0x8000)	<code>0x8000 → 20</code>	Object itself on stack; its internal pointer member points to the heap
<code>Student* s1 = new Student(20);</code>	<code>s1 = 0x9000</code>	Student object at 0x9000, whose age member points further to 0xA000 → 20	Both the object AND its internal data live on the heap

- 'Does changing 'a' affect *age?' → only if age = &a (same integer). If age = new int(a), they're independent copies.

5. Dynamic Safe Arrays

5.1 The Problem with Plain new[] Arrays

- Fixed size once created — to 'grow' it you must allocate a new bigger block, copy old elements, then delete the old block.
- No built-in bounds checking — accessing `arr[15]` on a 10-element array does not stop you; it just corrupts memory silently.
- Easy to forget `delete[]` → memory leak; easy to double-delete → crash.

5.2 The Fix: Wrap It in a Class

A 'safe' dynamic array is a class that owns a heap array internally and manages it responsibly through its constructor, destructor, and (critically) the Rule of Three — because copying an object that owns a heap array has exactly the same danger as copying the Student class below.

```
class SafeArray {
    int* data; int size;
public:
    // 1Constructor
    SafeArray(int n) { size = n; data = new int[n](); }
    // Destructor
    ~SafeArray() { delete[] data; }
    // bracket operator overloading
    int& operator[](int i) {
        if (i < 0 || i >= size) { cout << "Error: Index out of bounds!\n";
            exit(1); }
        return data[i]; }
    int getSize() { return size; }
```

```
};
```

5.3 Resizing Logic

```
void resize(int newSize) {  
    int* newData = new int[newSize];  
    int copyCount = min(size, newSize);  
  
    for (int i = 0; i < copyCount; i++) newData[i] = data[i];  
  
    delete[] data;    // release old block ONLY after copying  
    data = newData;  
    size = newSize;  
}
```

- The resize pattern (new → copy → delete[] old → reassign pointer → update size) is the core idea behind vector/ArrayList internally.
- Common planted bug: deleting the old array BEFORE copying its elements → data loss. Always copy first.

6. Copying Objects & the Rule of Three

6.1 The Problem: Default Copy Is Shallow

When a class owns heap memory via a raw pointer, C++'s default copy behaviour (compiler-generated copy constructor/assignment) just copies the POINTER VALUE — not the data it points to. This is a shallow copy.

```
class Student {  
    int* age;  
public:  
    Student(int a) { age = new int(a); }  
    ~Student() { delete age; }  
};  
  
Student s1(20);  
Student s2 = s1; // default copy → shallow copy!
```

	Shallow copy (default)	Deep copy (what we want)
What happens	s1.age and s2.age store the SAME address	s2 gets its OWN new heap block with the same value
Risk	Double free: both destructors delete the same address → crash	Each object safely owns and deletes its own memory

6.2 The Rule of Three

Rule: if a class manages a resource via a raw pointer and you need to write ONE of destructor / copy constructor / copy assignment operator, you almost certainly need to write ALL THREE.

Function	When it runs	Job
Destructor ~Student()	Object goes out of scope / deleted	Release the owned heap memory
Copy constructor Student(const Student&)	A NEW object is being created from an existing one	Allocate fresh memory + copy the value (deep copy)
Copy assignment operator=	Assigning into an ALREADY EXISTING object	Free old memory, then allocate fresh + copy (deep copy)

Quick rule to tell constructor vs assignment apart: if the destination didn't exist a moment ago → copy constructor. If it already existed → copy assignment.

```
Student s1(20);
Student s2 = s1; // s2 is being CREATED → copy constructor

Student s3(30);
s3 = s1;        // s3 already exists → copy assignment operator
```

6.3 The Complete, Safe Student Class

```
class Student {
    int* age;
public:
    Student(int a) {
        age = new int(a);
    }
    Student(const Student& other) { // Copy constructor
        age = new int(*other.age); // deep copy
    }
    Student& operator=(const Student& other) { // Copy assignment
        if (this != &other) { // guard: self-assignment
            delete age; // free OLD memory first
            age = new int(*other.age); // then deep copy
        }
        return *this; // enables chaining: a = b = c;
    }
    ~Student() { // Destructor
        delete age;
    }
};
```

6.4 The 'this' Pointer & Self-Assignment

- this is a pointer to the current object — inside operator=, 'this' is the destination, 'other' is the source.
- Self-assignment (s1 = s1;) means this == &other. Without the guard, you would delete age, then try to read *other.age — but other.age is the SAME (now-deleted) pointer → reading freed memory (undefined behaviour).
- The check if (this != &other) skips the delete+copy entirely when source and destination are the same object.

• Common planted bugs to spot: (1) missing 'delete age' in operator= → memory leak, (2) missing self-assignment check → crash on s1=s1, (3) copy constructor copying the pointer instead of using 'new' → shallow copy bug, (4) forgetting 'return *this' → breaks chained assignment.

- If a class has NO pointer/heap members (only plain ints, strings, etc.), the compiler-generated default copy is already safe — Rule of Three is not needed.

7. One-Page Master Summary

Concept	One-line takeaway
ADT	Defines WHAT operations exist; hides HOW they're implemented.
Pointer	A variable that stores an address; * dereferences to reach the value.
nullptr	Safe 'points to nothing' state — always check before dereferencing.
new / delete	Manually request/release heap memory; new[] must pair with delete[].
Memory leak	Heap memory that's lost (unreachable) because nothing deletes it.
Dynamic array	Runtime-sized array; 'safe' version wraps it in a class with bounds checks.
Shallow copy	Copies the pointer (address) — both objects share one resource. Dangerous.
Deep copy	Allocates a separate copy of the resource for the new object. Safe.
Rule of Three	Destructor + copy constructor + copy assignment — needed together whenever a class owns a raw-pointer resource.
this != &other	Self-assignment guard inside operator= — prevents deleting data you still need to read.